



WILLSIM

FICHE DE COURS · RESSOURCE PÉDAGOGIQUE GRATUITE

ROS 2 : premiers pas en robotique mobile

Découvrir ROS 2, le cadre logiciel de référence en robotique, du simulateur turtlesim jusqu'à un robot simulé qui cartographie une pièce et s'y déplace seul. Vous écrirez vos premiers nœuds Python, dont un arrêt d'urgence fondé sur un lidar, et verrez où l'IA s'insère.

NIVEAU

Débutant à
intermédiaire

DURÉE

8 h (4 séances de 2 h)

PRÉREQUIS

Bases de Linux
(terminal) et de
Python

OUTILS

ROS 2 Jazzy (Ubuntu
24.04), rclpy,
turtlesim, RViz2,
Gazebo, Nav2,
slam_toolbox

OBJECTIFS DE LA FICHE

À la fin de cette fiche, vous saurez :

- distinguer topics, services, actions et paramètres, et les observer avec les commandes `ros2` ;
- créer un paquet Python, le compiler avec `colcon` et lancer ses nœuds ;
- écrire un nœud `rclpy` qui publie, s'abonne et expose des paramètres ;
- expliquer la chaîne `map` → `odom` → `base_link` et lire une transformation avec `tf2` ;
- programmer un nœud d'arrêt d'urgence sur `LaserScan` et le valider par des critères mesurables ;
- lancer cartographie (`slam_toolbox`) et navigation (`Nav2`) en simulation, et situer l'apport de l'IA.

1. ROS 2 en cinq notions

ROS 2 (*Robot Operating System*) n'est pas un système d'exploitation : c'est un ensemble de bibliothèques, d'outils et de conventions qui permet de découper le logiciel d'un robot en programmes indépendants qui communiquent. Cette fiche utilise la distribution **Jazzy Jalisco**, version à support long (LTS) publiée en mai 2024 et maintenue jusqu'en mai 2029, sous Ubuntu 24.04 ; les commandes restent valables sur les distributions plus récentes, en remplaçant `jazzy` par leur nom.

1.1 Nœuds et topics

Un **nœud** est un programme qui remplit une fonction précise : piloter les moteurs, lire le lidar, planifier une trajectoire. Les nœuds échangent des **messages typés** sur des **topics**, selon le modèle publication-abonnement : le pilote du lidar publie sur `/scan` des messages `sensor_msgs/msg/LaserScan`, et tous les nœuds abonnés à `/scan` les reçoivent, sans que l'émetteur connaisse ses destinataires.

Les nœuds se découvrent automatiquement grâce au middleware DDS s'ils partagent le même `ROS_DOMAIN_ID` : en atelier, chaque binôme choisit une valeur différente entre 0 et 101 (`export ROS_DOMAIN_ID=12`). Chaque éditeur et chaque abonné déclare aussi une **qualité de service** (QoS), par exemple fiable (*reliable*) ou au mieux (*best effort*) : nous y reviendrons en section 5.

1.2 Services, actions et paramètres

Les quatre mécanismes de communication de ROS 2

Mécanisme	Principe	Exemple en robotique mobile	Outil
Topic	Flux continu, sans réponse, de N éditeurs vers M abonnés	<code>/scan</code> , <code>/cmd_vel</code> , <code>/odom</code>	<code>ros2 topic echo</code>
Service	Requête puis réponse, pour une opération courte	créer une tortue, enregistrer une carte	<code>ros2 service call</code>
Action	Tâche longue : retours intermédiaires, résultat, annulation	« rejoindre (x, y) » avec Nav2	<code>ros2 action send_goal</code>
Paramètre	Réglage d'un nœud, modifiable en marche	distance d'arrêt, vitesse maximale	<code>ros2 param set</code>

1.3 Découvrir les outils avec turtlesim

turtlesim est un simulateur 2D minimal fourni avec ROS 2. Dans chaque terminal, chargez ROS 2 (`source /opt/ros/jazzy/setup.bash`) puis essayez :

- `ros2 run turtlesim turtlesim_node` , puis `ros2 run turtlesim turtle_teleop_key` (pilotage au clavier) ;
- `ros2 node list` , `ros2 topic list -t` , `ros2 topic echo /turtle1/pose` et `rqt_graph` pour observer le système ;
- `ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.0, name: 'tortue2'}"` (service : une deuxième tortue) ;
- `ros2 action send_goal --feedback /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"` (action, avec retours intermédiaires) ;
- `ros2 param set /turtlesim background_r 150` (paramètre : couleur du fond).

2. Espace de travail, paquet et colcon

Votre code vit dans un **espace de travail** : un dossier dont le sous-dossier `src/` contient des **paquets**. Un paquet regroupe des nœuds, leurs dépendances (`package.xml`) et leur configuration d'installation (`setup.py` pour un paquet Python). L'outil **colcon** (installé avec `rosdep` par le paquet `ros-dev-tools`) compile les paquets de `src/` et crée les dossiers `build/` , `install/` et `log/` .

Terminal • créer et compiler le paquet `premiers_pas`

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
ros2 pkg create premiers_pas --build-type ament_python --license Apache-2.0 \
  --dependencies rclpy geometry_msgs sensor_msgs turtlesim
# Copier cercle.py et arret_urgence.py dans premiers_pas/premiers_pas/, puis ajouter
# à la liste 'console_scripts' des entry_points de setup.py :
#   'cercle = premiers_pas.cercle:main',
#   'arret_urgence = premiers_pas.arret_urgence:main',
cd ~/ros2_ws
# rosdep, une seule fois par machine : sudo rosdep init && rosdep update
rosdep install -i --from-path src --rosdistro jazzy -y # dépendances manquantes
colcon build --symlink-install
```

Dans un **nouveau terminal**, chargez l'espace de travail (`source ~/ros2_ws/install/setup.bash`) puis lancez `ros2 run premiers_pas cercle` ; ne compilez pas dans un terminal où il est déjà chargé. Avec `--symlink-install` , modifier un fichier Python ne demande pas de recompiler, contrairement à `setup.py` et `package.xml` .

3. Publisher et subscriber en Python

Le nœud `cercle.py` ci-dessous publie dix fois par seconde une consigne `geometry_msgs/msg/Twist` sur `turtle1/cmd_vel` : une vitesse linéaire v selon l'axe x de la tortue et une vitesse angulaire $\omega = v / R$ autour de l'axe z , d'où un cercle de rayon R . Il s'abonne aussi à `turtle1/pose` pour afficher la position.

Ses fonctions de rappel (*callbacks*) ne sont jamais appelées directement : `rclpy.spin()` les exécute quand le minuteur expire ou qu'un message arrive. L'entier `10` passé à `create_publisher` donne la QoS par défaut (« fiable ») avec une file de 10 messages. Les paramètres se règlent au lancement (`--ros-args -p rayon:=1.5`) ou en marche (`ros2 param set /cercle rayon 1.0`) ; écrivez `1.0` et non `1`, car le type est fixé par la valeur par défaut. Après Ctrl+C, la tortue s'arrête seule au bout d'une seconde : turtlesim applique un **délai de garde** (*watchdog*), comme les vrais robots.

premiers_pas/premiers_pas/cercle.py

```
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose

class Cercle(Node):
    def __init__(self):
        super().__init__('cercle')
        self.declare_parameter('vitesse', 1.0)  # vitesse linéaire (unités/s)
        self.declare_parameter('rayon', 2.0)    # rayon du cercle (unités)
        self.pub = self.create_publisher(Twist, 'turtle1/cmd_vel', 10)
        self.create_subscription(Pose, 'turtle1/pose', self.on_pose, 10)
        self.create_timer(0.1, self.on_timer)    # période de 0,1 s (10 Hz)

    def on_timer(self):
        v = self.get_parameter('vitesse').value
        r = max(self.get_parameter('rayon').value, 0.1)  # évite la division par 0
        cmd = Twist()
        cmd.linear.x = v  # avance selon l'axe x de la tortue
        cmd.angular.z = v / r  # omega = v / R, en rad/s
        self.pub.publish(cmd)

    def on_pose(self, pose):  # au plus un message par seconde
        self.get_logger().info(f'x={pose.x:.2f} y={pose.y:.2f}', throttle_duration_sec=1.0)

def main():
    rclpy.init()
    node = Cercle()
    try:
        rclpy.spin(node)  # exécute les callbacks jusqu'à Ctrl+C
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        rclpy.try_shutdown()
```

4. Repères et tf2

Un robot manipule de nombreux **repères** (*frames*) : celui du monde, celui du châssis, celui de chaque capteur. La bibliothèque **tf2** tient à jour l'arbre de ces repères, où chacun a au plus un parent, et calcule à tout instant la transformation (translation et rotation) entre deux d'entre eux. Selon la convention REP 103, l'axe x pointe vers l'avant du robot, y vers sa gauche et z vers le haut, et les angles positifs tournent dans le sens trigonométrique. Selon la REP 105, la chaîne de repères d'un robot mobile est `map` → `odom` → `base_link` (figure 1).

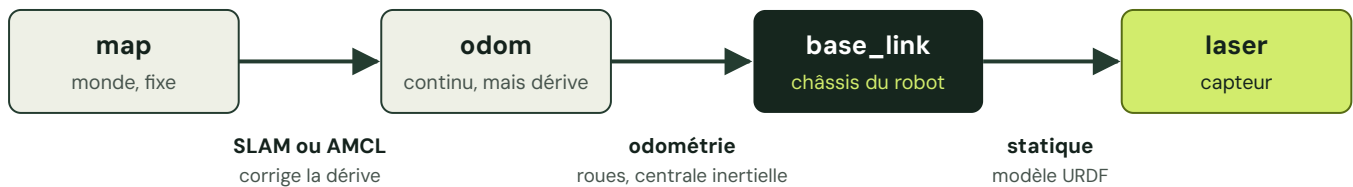


Figure 1 — Chaîne de repères d'un robot mobile (REP 105) ; sous chaque flèche, ce qui publie la transformation.

`odom` est continu mais dérive (glissement des roues, diamètre imprécis) ; la localisation corrige cette dérive en publiant `map` → `odom`, quitte à faire de petits sauts. La position du lidar, fixe, vient du modèle URDF via `robot_state_publisher`. Pour essayer sans robot :

- `ros2 run tf2_ros static_transform_publisher --x 0.1 --y 0 --z 0.2 --yaw 0 --pitch 0 --roll 0 --frame-id base_link --child-frame-id laser` déclare un lidar placé 10 cm devant et 20 cm au-dessus de `base_link` ;
- `ros2 run tf2_ros tf2_echo base_link laser` affiche cette transformation (translation 0,1 ; 0 ; 0,2) ; `ros2 run tf2_tools view_frames` enregistre l'arbre en PDF, et l'affichage *TF* de RViz2 le montre en 3D.

5. Un nœud d'arrêt d'urgence à partir d'un LaserScan

Un lidar 2D mesure des distances dans un plan horizontal en balayant un angle. Chaque balayage produit un message `sensor_msgs/msg/LaserScan` : la mesure n° i du tableau `ranges` est prise à l'angle `angle_min + i × angle_increment`, dans le repère du capteur. Seules les valeurs entre `range_min` et `range_max` sont valides ; selon la REP 117, `+inf` signifie « aucun écho », `-inf` « trop proche pour être mesuré » et `NaN` « mesure erronée ». La fonction suivante renvoie la distance du plus proche obstacle devant le robot (axe x du lidar orienté vers l'avant).

premiers_pas/premiers_pas/arret_urgence.py (1/2)

```

import math

import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan

def distance_devant(scan, demi_angle):
    """Plus petite distance mesurée dans le cône [-demi_angle, +demi_angle] (rad)."""
    d_min = math.inf
    for i, r in enumerate(scan.ranges):
        a = scan.angle_min + i * scan.angle_increment
        a = math.atan2(math.sin(a), math.cos(a)) # angle ramené dans [-pi, pi]
        valide = scan.range_min <= r <= scan.range_max or r == -math.inf # REP 117
        if abs(a) <= demi_angle and valide:
            d_min = min(d_min, r) # NaN et +inf sont ignorés
    return d_min

```

Le nœud s'intercale entre la source des consignes (clavier ou Nav2) et le robot : il relaie `cmd_vel_in` vers `cmd_vel`, mais annule la marche avant si un obstacle est à moins de `distance_arret` dans un cône de $\pm$ `demi_angle_deg`, ou si le dernier scan date de plus de 0,5 s (un capteur muet est un danger). Rotation et recul restent permis pour se dégager. Les deux paramètres sont lus au démarrage (`--ros-args -p distance_arret:=0.8`). Le scan est reçu avec la QoS `qos_profile_sensor_data` (*best effort*) : un abonné *reliable* ne recevrait rien d'un pilote qui publie en *best effort*, alors que l'inverse fonctionne.

```

class ArretUrgence(Node):
    """Relaie cmd_vel_in vers cmd_vel et bloque la marche avant devant un obstacle."""

    def __init__(self):
        super().__init__('arret_urgence')
        self.seuil = self.declare_parameter('distance_arret', 0.5).value # m
        self.demi_angle = math.radians(self.declare_parameter('demi_angle_deg', 30.0).value)
        self.bloque = False
        self.t_scan = -math.inf # instant du dernier scan (ns)
        self.pub = self.create_publisher(Twist, 'cmd_vel', 10)
        self.create_subscription(LaserScan, 'scan', self.on_scan, qos_profile_sensor_data)
        self.create_subscription(Twist, 'cmd_vel_in', self.on_cmd, 10)

    def on_scan(self, scan):
        self.t_scan = self.get_clock().now().nanoseconds
        bloque = distance_devant(scan, self.demi_angle) < self.seuil
        if bloque and not self.bloque: # un obstacle vient d'apparaître
            self.pub.publish(Twist()) # arrêt immédiat : vitesses nulles
            self.get_logger().warning('Obstacle devant : marche avant bloquée')
            self.bloque = bloque

    def on_cmd(self, cmd):
        age = (self.get_clock().now().nanoseconds - self.t_scan) * 1e-9 # en s
        if (self.bloque or age > 0.5) and cmd.linear.x > 0.0:
            cmd.linear.x = 0.0 # rotation et recul restent permis
        self.pub.publish(cmd)

def main():
    rclpy.init()
    node = ArretUrgence()
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        rclpy.try_shutdown()

```

ATTENTION · PASSAGE AU ROBOT RÉEL

- Ce nœud est une aide à la conduite, pas une fonction de sécurité : un robot réel doit avoir un **arrêt d'urgence matériel** (bouton coup-de-poing) qui coupe l'alimentation des moteurs, indépendamment du logiciel.
- Premiers essais roues décollées du sol, puis à 0,2 m/s au plus, dans une zone dégagée et sans personne sur la trajectoire.
- Batteries au lithium : chargeur adapté, charge surveillée sur support ininflammable, pas de décharge profonde, rebut si gonflée.

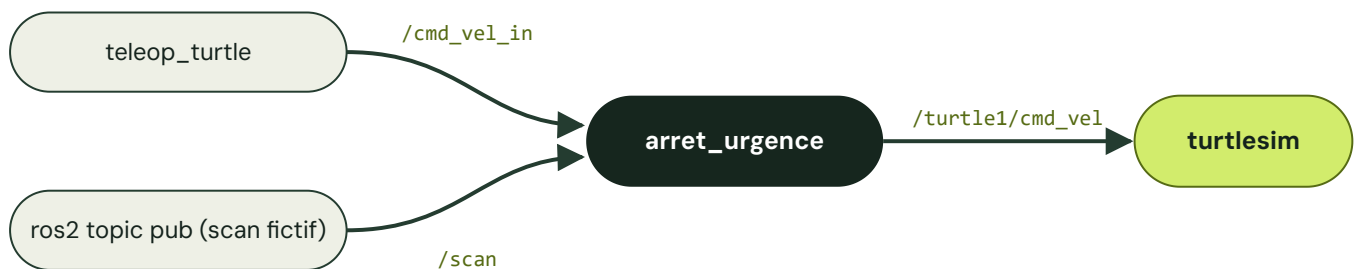


Figure 2 — Graphe de l'exercice : nœuds en pastilles, topics sur les flèches. Le nœud lancé par `turtle_teleop_key` s'appelle `teleop_turtle`.

EXERCICE GUIDÉ · BLOQUER LA TORTUE DEVANT UN OBSTACLE VIRTUEL

Faute de lidar, on publie un scan fictif de cinq mesures (de -90° à $+90^\circ$, pas de 45°) et l'on intercale `arret_urgence` entre le clavier et turtlesim (figure 2). Quatre terminaux, espace de travail chargé :

```
ros2 run turtlesim turtlesim_node
ros2 run premiers_pas arret_urgence --ros-args -r cmd_vel:=turtle1/cmd_vel
ros2 run turtlesim turtle_teleop_key --ros-args -r turtle1/cmd_vel:=cmd_vel_in
ros2 topic pub -r 10 /scan sensor_msgs/msg/LaserScan "{header: {frame_id: laser},
  angle_min: -1.5708, angle_max: 1.5708, angle_increment: 0.7854,
  range_min: 0.1, range_max: 10.0, ranges: [2.0, 2.0, 2.0, 2.0, 2.0]}"
```

1. Mesures à 2,0 m : la tortue avance-t-elle ? `rqt_graph` montre-t-il les liaisons de la figure 2 ?
2. Republiez avec `ranges: [2.0, 2.0, 0.3, 2.0, 2.0]` (obstacle droit devant). Que font les flèches haut, gauche et bas ?
3. Mettez 0,3 en dernière position (obstacle à $+90^\circ$). L'avance est-elle bloquée ? Pourquoi ?
4. Arrêtez `ros2 topic pub` (Ctrl+C) et essayez d'avancer.
5. Un robot roule à $v = 0,5$ m/s, réagit en $t_r = 0,2$ s et freine à $a = 1$ m/s². Calculez $d = v t_r + v^2 / (2a)$: 0,5 m suffit-il ?

Critères de réussite : liaisons conformes à la figure 2 ; avance refusée dans les cas 2 et 4 seulement ; rotation toujours possible ; un seul avertissement à l'apparition de l'obstacle.

Corrigé : cas 2, $0,3 \text{ m} < 0,5 \text{ m}$: avance annulée, rotation et recul permis. Cas 3, $+90^\circ$ est hors du cône de $\pm 30^\circ$: rien n'est bloqué. Cas 4, scan périmé après 0,5 s : avance refusée. Cas 5, $d = 0,1 + 0,125 = 0,225 \text{ m}$; même en ajoutant la distance du lidar à l'avant du robot, 0,5 m laisse une marge à cette vitesse, mais pas à 1 m/s ($d = 0,7 \text{ m}$). Prolongements : couloir de la largeur du robot, minuteur qui publie une consigne nulle si le scan est périmé.

6. Cartographier et naviguer : SLAM et Nav2

Le **SLAM** (*Simultaneous Localization And Mapping*) construit une carte tout en s'y localisant. **slam_toolbox**, utilisé par défaut avec Nav2, recalcule chaque scan sur les précédents et corrige son graphe de poses lors des **fermetures de boucle** (retour dans un lieu connu). Il produit une grille d'occupation (`nav_msgs/msg/OccupancyGrid`) aux cellules libres, occupées ou inconnues.

Nav2 est la pile de navigation de ROS 2. Sur une carte connue, AMCL (filtre particulaire) localise le robot ; des cartes de coûts « gonflent » les obstacles d'une marge ; un planificateur calcule le chemin, un contrôleur le suit, un arbre de comportements orchestre le tout. On le commande par l'action `navigate_to_pose`.

- Installer : `sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup ros-jazzy-nav2-minimal-tb*` ;
- cartographier : `ros2 launch nav2_bringup tb3_simulation_launch.py slam:=True headless:=False` lance Gazebo, un TurtleBot 3 simulé, **slam_toolbox**, Nav2 et RViz2 ; explorez la pièce avec le bouton *Nav2 Goal* ;
- enregistrer la carte (`carte.pgm` et `carte.yaml`) avec `ros2 run nav2_map_server map_saver_cli -f ~/carte`, puis naviguer : relancez sans `slam:=True` avec `map:=$HOME/carte.yaml`, donnez la position de départ (2D *Pose Estimate*) et un objectif.

7. Où intervient l'IA ?

Localisation, cartographie et planification reposent ici sur des algorithmes classiques, éprouvés et explicables. L'IA (surtout l'apprentissage profond) intervient d'abord dans la **perception**.

Approche classique et apport de l'IA

Fonction	Approche classique	Apport de l'IA
Détection d'obstacles	Lidar 2D : tout ce qui coupe son plan	Caméra et réseau de détection : obstacles hors de ce plan (plateau de table, objet au sol)
Reconnaissance	Aucune : un obstacle reste un obstacle	Distinguer une personne d'un meuble pour adapter marge ou vitesse
Commande	Contrôleurs de Nav2 (DWB, MPPI...)	Politiques apprises par renforcement en simulation

En pratique, un nœud de perception publie ses détections (`vision_msgs/msg/Detection2DArray`) ; situées dans l'espace grâce à la profondeur et à `tf2`, elles alimentent une couche de la carte de coûts de Nav2. Sur le calculateur embarqué, la latence du modèle doit être mesurée.

À RETENIR

- Topic pour un flux, service pour une question-réponse, action pour une tâche longue et annulable, paramètre pour un réglage ; `rqt_graph` et `ros2 topic echo` pour diagnostiquer.
- Un détecteur appris peut manquer un obstacle (faux négatif) : il enrichit la perception sans remplacer les barrières déterministes (arrêt d'urgence matériel, nœud d'arrêt sur lidar, vitesse limitée). Maîtrisez d'abord la chaîne classique, greffez-y l'IA ensuite.

Pour aller plus loin

- Documentation officielle de ROS 2 Jazzy (installation, concepts) — <https://docs.ros.org/en/jazzy/>
- Tutoriels officiels (outils en ligne de commande, `rclpy`, `tf2`) — <https://docs.ros.org/en/jazzy/Tutorials.html>
- REP 105, repères des plateformes mobiles — <https://reps.openrobotics.org/rep-0105/> et REP 117, mesures de distance — <https://reps.openrobotics.org/rep-0117/>
- Documentation de Nav2 — <https://docs.nav2.org/>
- `slam_toolbox`, dépôt officiel — https://github.com/SteveMacenski/slam_toolbox
- Documentation de Gazebo — <https://gazebo.org/docs/>
- Page du cours sur willsim.fr — <https://willsim.fr/cours/ros2-premiers-pas.html>

Fiche rédigée par l'équipe pédagogique WILLSIM, association loi 1901 (RNA W842013616) qui promeut l'usage de l'intelligence artificielle dans l'ingénierie et la mécanique. Version 1.0 – octobre 2026.

© 2026 WILLSIM — Licence Creative Commons BY-NC-SA 4.0 (creativecommons.org/licenses/by-nc-sa/4.0/deed.fr) : vous pouvez copier, diffuser et adapter cette fiche à des fins non commerciales, en citant WILLSIM et en partageant vos adaptations sous la même licence.

Une erreur, une suggestion ? Écrivez à contact@willsim.fr · willsim.fr